



DANI CABALLERO MÉLICH

GAMEPLAY / SYSTEMS PROGRAMMER

Barcelona · on-site, hybrid or remote
EU citizen, no sponsorship needed
dani.caballero.m@gmail.com
+34 644 974 715
linkedin.com/in/dani-caballero-melich
danimelich.com

ENGINES & LANGUAGES

Production C# · Unity · Godot · GDScript · Kotlin (Android plugin) · Python
Platforms Android · iOS
Tools Git · PRs and code review · CI · Figma

GAMEPLAY SYSTEMS

Real-time combat · ability and card systems · state machines · staged damage-resolution pipeline · enemy behaviour · roguelike generation · game feel · UI systems

META & ECONOMY

Gacha odds · pity timer · featured 50/50 · five-currency economy · upgrade ladders · talent trees · sources and sinks · premium-currency placement · task economies · balance tuning · simulation-based balancing

LIVE OPS & F2P

Cloud save and sync · server-driven remote config · leaderboards · push · analytics and run telemetry · rewarded video · IAP integration · localisation pipeline · signed release builds

I build gameplay and meta systems for mobile, and I ship them. I was one of the core programming trio on **Stepland**, live on Google Play and the App Store, and I carried **Dungeon Heroes**, a Unity action roguelike, alone to signed release builds — combat, a five-currency meta-economy, versioned save migration and live-ops.

TITLES

- **Stepland** · Godot · Android + iOS · live on Google Play and the App Store
Somni Game Studios — one of the core programming trio in a larger team.
Habit-tracking lifestyle game: real-world walking grows a virtual pet. Mine: the step-counting plugin, app boot, cloud sync, remote config, streaks, exploration.
- **Dungeon Heroes** · Unity · Android · signed release builds, store-ready
Solo: design, code, art, UI, release engineering. Real-time action roguelike on a cycling four-card hand, with a gacha-and-upgrade meta layer.
- **Degen Crawler** · C# · browser · degendungeons.com/crawler
Gameplay programmer, three-person studio. Roguelike where players wager crypto on each run. Mine: ~20 relics, the Seashore biome, and the balance agent.

EXPERIENCE

Somni Game Studios · Technical Designer & Gameplay Programmer

2025 – 2026 · Barcelona, remote · Stepland (Godot, Android + iOS)

- **I replaced the third-party step counter with an in-house Android plugin, built in a week in Kotlin, a language I had never written.** Step tracking ran through Google Health Connect: players had to configure a second app before Stepland worked, and a misconfiguration failed silently — "connected", reading zero steps. I proposed the cut, wrote a plugin that reads the Android pedometer directly, and added a WorkManager job that keeps counting while the app is closed. One permission prompt at first launch, and stable since.
- **I designed and built the two-tier task economy** — long-run progress goals and daily tasks — as declarative task resources with staged rewards and premium-currency placement, plus the task UI. The team grows retention content by authoring data, not new systems.
- **I rebuilt the tutorial as 82 declarative steps and landed it on the opening day of the 4YFN showcase** — a library of polymorphic step types under a sequence runner with resume-from-index and telemetry, built by driving coding-agent sessions against the deadline.
- **I owned app boot, cloud sync and server-driven remote config**, plus the streaks retention feature end to end with its test suite and the localisation pipeline — the plumbing that lets a live title change content and keep player state across devices. Day to day inside the dev team: daily meetings, PRs and code review.
- **I greyboxed features before art touched them.** Screens were reaching full fidelity before their requirements settled, so I blocked out each screen and flow — Figma, or plain Godot panels with working behaviour — and handed the structure over to be dressed. The streak screen still ships in the shape I gave it.

Freelance · Pixel art and contract development

2024 – 2025 · commissioned pixel art for indie developers through Upwork

ARCHITECTURE

Service layer · dependency injection · data-driven configuration · versioned save migration · schema versioning · unit tests

ALSO

Self-taught from seventeen: C#, Unity, and 15+ prototypes in six years before turning professional.

Grandmaster, EU — peak ~rank 300 in Teamfight Tactics. Where the balance agent's heuristics came from.

Gamescom · Godot Fest · BCN Game Fest · 4YFN — on the floor for a studio.

Catalan and Spanish native · English fluent.

Purple Pwny Studios · Gameplay Programmer

2023 – 2024 · remote, US studio · Unity and C# · Degen Crawler and Degen Drafts

- **I designed and implemented ~20 relics inside a 65,000-line C# codebase I had not written.** Each brief was a name and an icon; the effect and its balance were mine. Where the engine could not express a relic I extended it — a flat-price shop cell, a reason code threaded through every currency change, sell pricing — so each one shipped as a vertical slice rather than a special case.
- **I authored the Seashore biome:** its six enemies, the Tenderize status that runs through them, and the boss ability — a two-mode attack that buffs, or hits double once buffed, and drops the buff on a critical hit, so the fight has a state the player can read and punish.
- **I built the balance agent that modelled strong-human play,** one of three reference points for tuning a game where players wager real money and the economy has to survive good players. A hand-authored policy — a rule tree per landmark type with explicit thresholds over a generated per-class relic value table — run through Monte Carlo simulation, calibrated against my own play at the top of the leaderboard.
- **I built Degen Drafts' Unity client** — the studio's second title — its data architecture from scratch, a seeded deterministic RNG, and a behaviour generator that works backwards from a round's pre-determined winner to drink times and distractions, so a fixed outcome still reads as a contest.

DUNGEON HEROES — THE SYSTEMS

Unity · Android · solo · ~47,500 lines of C#, 174 spells, 52 status effects, 38 relics

- **The meta-economy layer.** I built chest openings with published odds, a pity timer and a featured 50/50; a smithing star ladder; talents and loadouts, all spending across five currencies. Odds, upgrade costs and rewards are authored in one data layer rather than scattered through gameplay code.
- **Versioned save migration.** Profiles migrate v1 → v2 → v3 on load; I made each step idempotent so a partially-migrated save re-migrates safely, and a guard refuses a profile written by a newer build rather than corrupting it. Adding a meta system stopped meaning invalidating a player's progress.
- **Real-time combat on a cycling four-card hand.** Cards refill on a timer, energy regenerates continuously, casts lock the hero for the length of their animation (0.42–1.5s), and enemies act one at a time through a queue with a global gap — I author difficulty as readable timing, not raw damage. The parry window sits on a 0.6–0.7s flash lead, re-tuned after play showed my earlier constant sat on the unreactable side.
- **Run telemetry built for balance.** Every run writes a schema-versioned record, enums crossing the wire by name so reordering them cannot silently remap old data — and the offline analyzer refuses my own editor and dev-machine runs as balance signal, behind a banner that cannot be suppressed, so tuning moves on play data, not on mine.
- **Live ops and release.** Google Play identity with cloud save and sync; Leaderboards, Analytics, Push, rewarded video and IAP SDKs integrated behind the meta layer. I carried it to signed Android release builds (8 artifacts, v0.4.4 AAB → v0.5.6 APK), store-ready with privacy policy, data-deletion pages and a release runbook.
- **Production tooling.** I built the multi-agent pipeline that turns a written UI spec into a finished Unity screen, backed by a 6-job CI — its screens ship in the game.

Case studies and code walkthroughs at danimelich.com